PART 1: BIG DATA CONTEXT (CONCEPTUAL)

BIG DATA REFLECTION:

Big data originates from various **sources**, including **social media platforms** (**Facebook, Twitter**), **IoT devices** (**smart appliances**, **fitness trackers**), **transaction systems** (**e-commerce**, **banking**), and **log data** from **websites** and **mobile apps**. These sources generate vast amounts of data in different formats, such as **text, images, videos**, and **structured records**. The defining characteristics of **big data** are known as the **5Vs**: **Volume** (large data size), **Velocity** (rapid data generation), **Variety** (diverse data types), **Veracity** (data accuracy), and **Value** (meaningful insights derived). Understanding these **sources** and **characteristics** helps organizations leverage **big data** for **decision-making** and **innovation**.

DATA WRANGLING:

Data wrangling and **cleaning** are essential because raw data from various **sources** is often **incomplete, inconsistent**, and **noisy**. It may contain **missing values, duplicate entries**, incorrect formats, and **outliers**, which can negatively impact **analysis accuracy**. **Wrangling** involves acquiring data, **cleaning** it by correcting errors, filling missing values, and transforming it into a **consistent format**, ensuring data is **accurate, complete**, and **ready for analysis**. Additionally, **data cleaning** includes **normalization** or **standardization**, ensuring all data is in a consistent format and scale, making it suitable for **advanced analytics** and **machine learning**. **Cleaned data** allows accurate **analysis, visualization**, and meaningful insights. Without proper **wrangling**, data scientists may waste time fixing **data quality issues**, leading to **misleading conclusions** and **poor decision-making**.

Potential Big Data Sources

Social Media Platforms:

Social media platforms like **Twitter, Facebook, Instagram, and LinkedIn** generate an enormous amount of **user-generated content** in the form of **text posts, images, videos, comments, and reactions**. This data is not only vast but also highly dynamic, constantly being updated in real time. Data scientists can leverage this data for **sentiment analysis** (understanding user opinions), **trend**

tracking (identifying popular topics or hashtags), **social network analysis** (examining user connections and influence), and **personalized content recommendations** (suggesting relevant posts or ads).

E-commerce Websites:

Platforms such as **Amazon, eBay, and Alibaba** produce a wealth of data, including **transactional records** (order history, payment details), **product information** (prices, categories, descriptions), and **customer behavior data** (clicks, search queries, product reviews). This data is crucial for building **recommender systems** (suggesting products to users), **sales forecasting** (predicting future sales trends), **customer segmentation** (grouping users based on behavior), and **sentiment analysis on reviews** (understanding customer feedback).

IoT Sensor Networks:

Internet of Things (IoT) devices, including **smart home devices (thermostats, smart locks), industrial sensors (machine monitors), and wearable gadgets (fitness trackers)**, generate continuous real-time data. This can include **temperature readings, device status, machine performance metrics, and user health data**. In data science, this data is used for **predictive maintenance** (identifying equipment issues before failure), **anomaly detection** (spotting unusual patterns), **energy optimization** (efficient power usage), and **health monitoring** (tracking vital signs for early diagnosis).

PART 2: DATASET SELECTION AND SETUP

```
import pandas as pd
```

```
from google.colab import drive
drive.mount('/content/drive')
```

```
Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mour
```

PART 3: DATA LOADING AND INSPECTION

1.Load the CSV

```
path='/content/drive/MyDrive/dataset/car.simulated.csv'
df=pd.read_csv(path)
```

```
print(df)
```

```
           ID       Brand    Model  Year    Price   Mileage       City  \
0           1  Volkswagen  Model B  2001  25000.0   50000.0    Cologne
1           2    Mercedes  Model D  1994  10000.0  200000.0     Munich
2           3        Ford  Model B  2016  15000.0       NaN    Hamburg
3           4    Mercedes  Model D  2004  25000.0  150000.0    Hamburg
4           5    Mercedes  Model B  2015  20000.0  150000.0  Frankfurt
...       ...         ...      ...   ...      ...       ...        ...
29995   29996         BMW  Model C  2020  10000.0  200000.0     Berlin
29996   29997        Audi  Model C  2018  15000.0  150000.0    Hamburg
29997   29998         BMW  Model B  2001   5000.0   50000.0    Hamburg
29998   29999        Audi  Model B  1998  15000.0   50000.0  Frankfurt
29999   30000    Mercedes  Model C  2018  15000.0  100000.0    Hamburg

         Registration Date
0      2014-02-05 00:00:00
1      2013-02-06 00:00:00
2      2021-03-12 00:00:00
3      2018-02-15 00:00:00
4      2011-06-30 00:00:00
...                    ...
29995  2016-07-10 00:00:00
29996  2022-11-13 00:00:00
29997  2011-12-12 00:00:00
29998  2010-01-31 00:00:00
29999  2022-01-29 00:00:00

[30000 rows x 8 columns]
```

2. Initial Exploration

```
df.head(5)
```

	ID	Brand	Model	Year	Price	Mileage	City	Registration Date
0	1	Volkswagen	Model B	2001	25000.0	50000.0	Cologne	2014-02-05 00:00:00
1	2	Mercedes	Model D	1994	10000.0	200000.0	Munich	2013-02-06 00:00:00
2	3	Ford	Model B	2016	15000.0	NaN	Hamburg	2021-03-12 00:00:00
3	4	Mercedes	Model D	2004	25000.0	150000.0	Hamburg	2018-02-15 00:00:00
4	5	Mercedes	Model B	2015	20000.0	150000.0	Frankfurt	2011-06-30 00:00:00

Next steps: Generate code with df | View recommended plots | New interactive sheet

```
df.tail(5)
```

	ID	Brand	Model	Year	Price	Mileage	City	Registration Date
29995	29996	BMW	Model C	2020	10000.0	200000.0	Berlin	2016-07-10 00:00:00
29996	29997	Audi	Model C	2018	15000.0	150000.0	Hamburg	2022-11-13 00:00:00
29997	29998	BMW	Model B	2001	5000.0	50000.0	Hamburg	2011-12-12 00:00:00

```
df.shape
```

```
(30000, 8)
```

```
df.describe
```

pandas.core.generic.NDFrame.describe
def describe(percentiles=None, include=None, exclude=None) -> Self

/usr/local/lib/python3.11/dist-packages/pandas/core/generic.py
Generate descriptive statistics.

Descriptive statistics include those that summarize the central tendency, dispersion and shape of a
dataset's distribution, excluding ``NaN`` values.

```
df.info
```

pandas.core.frame.DataFrame.info
def info(verbose: bool | None=None, buf: WriteBuffer[str] | None=None, max_cols: int | None=None, memory_usage: bool | str | None=None, show_counts: bool | None=None) -> None

/usr/local/lib/python3.11/dist-packages/pandas/core/frame.py
Print a concise summary of a DataFrame.

This method prints information about a DataFrame including
the index dtype and columns, non-null values and memory usage.

PART 4: DATA CLEANING AND WRANGLING

1. Handling Missing Values

```
missing_values = df.isnull().sum()
```

```
missing_values = missing_values[missing_values > 0]
```

```
print(missing_values)
```

```
Price      5054
Mileage    5978
dtype: int64
```

```
median_price = df['Price'].median()
df['Price'].fillna(median_price, inplace=True)
print(median_price)
```

```
15000.0
<ipython-input-33-3c8b4d29f5cb>:2: FutureWarning: A value is trying to be set on a copy
The behavior will change in pandas 3.0. This inplace method will never work because the

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col

  df['Price'].fillna(median_price, inplace=True)
```

```
mean_mileage = df['Mileage'].mean()
df['Mileage'].fillna(mean_mileage, inplace=True)
print(mean_mileage)
```

```
124920.90583631671
<ipython-input-34-a0a9edb901ff>:2: FutureWarning: A value is trying to be set on a copy
The behavior will change in pandas 3.0. This inplace method will never work because the

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col

  df['Mileage'].fillna(mean_mileage, inplace=True)
```

```
print("Missing Values After Handling:\n", df.isnull().sum())
```

```
Missing Values After Handling:
 ID                   0
Brand                0
Model                0
Year                 0
Price                0
Mileage              0
City                 0
Registration Date    0
```

```
dtype: int64
```

2. Handling Duplicates

```
duplicate_count = df.duplicated().sum()
print(f"Total Duplicate Rows Before Dropping: {duplicate_count}")
```

```
Total Duplicate Rows Before Dropping: 51
```

```
df.drop_duplicates(inplace=True)

new_duplicate_count = df.duplicated().sum()
removed_count = duplicate_count - new_duplicate_count
print(f"Total Duplicate Rows Removed: {removed_count}")
print(f"Total Duplicate Rows After Dropping: {new_duplicate_count}")
```

```
Total Duplicate Rows Removed: 51
Total Duplicate Rows After Dropping: 0
```

3. Data Type Conversions

```
print("Data Types Before Conversion:\n", df.dtypes)
```

```
Data Types Before Conversion:
 ID                     int64
Brand                 object
Model                 object
Year                   int64
Price                float64
Mileage              float64
City                  object
Registration Date     object
dtype: object
```

```
# Converting 'Price' column to numeric
df['Price'] = pd.to_numeric(df['Price'], errors='coerce')
print(df['Price'].dtype)
# Converting 'Mileage' column to numeric
df['Mileage'] = pd.to_numeric(df['Mileage'], errors='coerce')
print(df['Mileage'].dtype)
```

```
float64
float64
```

```
df.rename(columns={'Mileage': 'Odometer', 'Price': 'Cost'}, inplace=True)
print(df.head(5))
```

```
   ID       Brand    Model  Year     Cost       Odometer       City  \
0   1  Volkswagen  Model B  2001  25000.0   50000.000000    Cologne
1   2    Mercedes  Model D  1994  10000.0  200000.000000     Munich
2   3        Ford  Model B  2016  15000.0  124920.905836    Hamburg
3   4    Mercedes  Model D  2004  25000.0  150000.000000    Hamburg
4   5    Mercedes  Model B  2015  20000.0  150000.000000  Frankfurt

     Registration Date
0  2014-02-05 00:00:00
1  2013-02-06 00:00:00
2  2021-03-12 00:00:00
3  2018-02-15 00:00:00
4  2011-06-30 00:00:00
```

4. Data Transformation

```
df['Cost_Per_KM'] = df['Cost'] / df['Odometer']
print(df.head())
```

```
   ID       Brand    Model  Year     Cost       Odometer       City  \
0   1  Volkswagen  Model B  2001  25000.0   50000.000000    Cologne
1   2    Mercedes  Model D  1994  10000.0  200000.000000     Munich
2   3        Ford  Model B  2016  15000.0  124920.905836    Hamburg
3   4    Mercedes  Model D  2004  25000.0  150000.000000    Hamburg
4   5    Mercedes  Model B  2015  20000.0  150000.000000  Frankfurt

     Registration Date  Cost_Per_KM
0  2014-02-05 00:00:00     0.500000
1  2013-02-06 00:00:00     0.050000
2  2021-03-12 00:00:00     0.120076
3  2018-02-15 00:00:00     0.166667
4  2011-06-30 00:00:00     0.133333
```

```
import pandas as pd
```

```
if 'Date' in df.columns:
    df['Year'] = pd.to_datetime(df['Date'], errors='coerce').dt.year
else:
    print("No 'Date' column found. Please ensure the date column exists and is properly form
```

```
# 2. Combining categories
def categorize_fuel(fuel_type):
    if pd.isna(fuel_type):  # Handling NaN or missing values
        return 'Unknown'
    elif fuel_type in ['Diesel', 'Petrol']:
        return 'Diesel/Petrol'
    elif fuel_type == 'Electric':
        return 'Electric'
    else:
        return 'Other'

if 'FuelType' in df.columns:
    df['FuelType_Category'] = df['FuelType'].apply(categorize_fuel)
else:
    print("No 'FuelType' column found. Please replace 'FuelType' with the actual column name

# 3. String cleaning:
if 'Model' in df.columns:
    df['Model'] = df['Model'].str.strip()
else:
    print("No 'Model' column found. Please replace 'Model' with the actual column name.")

print(df)
```

```
No 'Date' column found. Please ensure the date column exists and is properly formatted.
No 'FuelType' column found. Please replace 'FuelType' with the actual column name.
          ID       Brand    Model  Year     Cost       Odometer       City  \
0          1  Volkswagen  Model B  2001  25000.0   50000.000000    Cologne
1          2    Mercedes  Model D  1994  10000.0  200000.000000     Munich
2          3        Ford  Model B  2016  15000.0  124920.905836    Hamburg
3          4    Mercedes  Model D  2004  25000.0  150000.000000    Hamburg
4          5    Mercedes  Model B  2015  20000.0  150000.000000  Frankfurt
...      ...         ...      ...   ...      ...            ...        ...
29995  29996         BMW  Model C  2020  10000.0  200000.000000     Berlin
29996  29997        Audi  Model C  2018  15000.0  150000.000000    Hamburg
29997  29998         BMW  Model B  2001   5000.0   50000.000000    Hamburg
29998  29999        Audi  Model B  1998  15000.0   50000.000000  Frankfurt
29999  30000    Mercedes  Model C  2018  15000.0  100000.000000    Hamburg

         Registration Date  Cost_Per_KM
0      2014-02-05 00:00:00     0.500000
1      2013-02-06 00:00:00     0.050000
2      2021-03-12 00:00:00     0.120076
3      2018-02-15 00:00:00     0.166667
4      2011-06-30 00:00:00     0.133333
...                    ...          ...
29995  2016-07-10 00:00:00     0.050000
29996  2022-11-13 00:00:00     0.100000
29997  2011-12-12 00:00:00     0.100000
29998  2010-01-31 00:00:00     0.300000
29999  2022-01-29 00:00:00     0.150000
```

```
[29949 rows x 9 columns]
```

```
if 'Registration Date' in df.columns:
    df['RegistrationYear'] = pd.to_datetime(df['Registration Date'], errors='coerce').dt.yea
else:
    print("Column 'RegistrationDate' not found in the DataFrame. Please check your data.")
    # Handle the missing column
print(df.head())
```

```
   ID       Brand    Model  Year     Cost       Odometer       City  \
0   1  Volkswagen  Model B  2001  25000.0   50000.000000    Cologne
1   2    Mercedes  Model D  1994  10000.0  200000.000000     Munich
2   3        Ford  Model B  2016  15000.0  124920.905836    Hamburg
3   4    Mercedes  Model D  2004  25000.0  150000.000000    Hamburg
4   5    Mercedes  Model B  2015  20000.0  150000.000000  Frankfurt

     Registration Date  Cost_Per_KM  RegistrationYear
0  2014-02-05 00:00:00     0.500000              2014
1  2013-02-06 00:00:00     0.050000              2013
2  2021-03-12 00:00:00     0.120076              2021
3  2018-02-15 00:00:00     0.166667              2018
4  2011-06-30 00:00:00     0.133333              2011
```

```
def price_to_category(price):
    if pd.isna(price):
        return 'Unknown'
    elif price < 10000:
        return 'Budget'
    elif price < 20000:
        return 'Mid-Range'
    else:
        return 'Premium'

df['PriceCategory'] = df['Cost'].apply(price_to_category)
print(df.head())
```

```
   ID       Brand    Model  Year     Cost       Odometer       City  \
0   1  Volkswagen  Model B  2001  25000.0   50000.000000    Cologne
1   2    Mercedes  Model D  1994  10000.0  200000.000000     Munich
2   3        Ford  Model B  2016  15000.0  124920.905836    Hamburg
3   4    Mercedes  Model D  2004  25000.0  150000.000000    Hamburg
4   5    Mercedes  Model B  2015  20000.0  150000.000000  Frankfurt

     Registration Date  Cost_Per_KM  RegistrationYear PriceCategory
0  2014-02-05 00:00:00     0.500000              2014       Premium
1  2013-02-06 00:00:00     0.050000              2013     Mid-Range
2  2021-03-12 00:00:00     0.120076              2021     Mid-Range
3  2018-02-15 00:00:00     0.166667              2018       Premium
4  2011-06-30 00:00:00     0.133333              2011       Premium
```

```
df['City'] = df['City'].str.strip()
print(df.head())
```

```
   ID       Brand    Model  Year     Cost       Odometer       City  \
0   1  Volkswagen  Model B  2001  25000.0   50000.000000    Cologne
1   2    Mercedes  Model D  1994  10000.0  200000.000000     Munich
2   3        Ford  Model B  2016  15000.0  124920.905836    Hamburg
3   4    Mercedes  Model D  2004  25000.0  150000.000000    Hamburg
4   5    Mercedes  Model B  2015  20000.0  150000.000000  Frankfurt

     Registration Date  Cost_Per_KM  RegistrationYear PriceCategory
0  2014-02-05 00:00:00     0.500000              2014       Premium
1  2013-02-06 00:00:00     0.050000              2013     Mid-Range
2  2021-03-12 00:00:00     0.120076              2021     Mid-Range
3  2018-02-15 00:00:00     0.166667              2018       Premium
4  2011-06-30 00:00:00     0.133333              2011       Premium
```

```
df['ModelCleaned'] = df['Model'].astype(str).str.replace(r'[^a-zA-Z0-9\s]', '', regex=True)
print(df.head())
```

```
   ID       Brand    Model  Year     Cost       Odometer       City  \
0   1  Volkswagen  Model B  2001  25000.0   50000.000000    Cologne
1   2    Mercedes  Model D  1994  10000.0  200000.000000     Munich
2   3        Ford  Model B  2016  15000.0  124920.905836    Hamburg
3   4    Mercedes  Model D  2004  25000.0  150000.000000    Hamburg
4   5    Mercedes  Model B  2015  20000.0  150000.000000  Frankfurt

     Registration Date  Cost_Per_KM  RegistrationYear PriceCategory  \
0  2014-02-05 00:00:00     0.500000              2014       Premium
1  2013-02-06 00:00:00     0.050000              2013     Mid-Range
2  2021-03-12 00:00:00     0.120076              2021     Mid-Range
3  2018-02-15 00:00:00     0.166667              2018       Premium
4  2011-06-30 00:00:00     0.133333              2011       Premium

  ModelCleaned
0      Model B
1      Model D
2      Model B
3      Model D
4      Model B
```

5. Filtering and Subsetting

```
# Filter for cars with 'Cost' greater than 20000 and 'Odometer' less than 50000
filtered_df = df[(df['Cost'] > 20000) & (df['Odometer'] < 50000)]
print("High Cost, Low Mileage Cars:\n", filtered_df)
```

```
High Cost, Low Mileage Cars:
 Empty DataFrame
Columns: [ID, Brand, Model, Year, Cost, Odometer, City, Registration Date, Cost_Per_KM,
Index: []
```

PART 5: PYTHON BASICS REVIEW

Use of loops

```
def categorize_prices(price_list):
    categories = []
    for price in price_list:
        if price < 5000:
            categories.append('Low')
        elif price < 15000:
            categories.append('Medium')
        elif price < 30000:
            categories.append('High')
        else:
            categories.append('Luxury')
    return categories

# Fill missing costs with 0 before categorization
df['Cost'] = df['Cost'].fillna(0)

df['price_segment'] = categorize_prices(df['Cost'].tolist())


print(df[['Cost', 'price_segment']].head())
```

```
       Cost price_segment
0  25000.0          High
1  10000.0        Medium
2  15000.0          High
3  25000.0          High
4  20000.0          High
```

List, Tuple, and Dictionary Usage

List

```
rows_to_append = []
for index, row in df.iterrows():
    if row['Cost'] > 25000 and row['Odometer'] < 20000:
        rows_to_append.append(row)

new_df = df[(df['Cost'] > 25000) & (df['Odometer'] < 20000)]
print(new_df)
```

```
Empty DataFrame
Columns: [ID, Brand, Model, Year, Cost, Odometer, City, Registration Date, price_segment
Index: []
```

Tuple

```
import numpy as np

# Function returns a tuple of mean, median, and max
def cost_summary(costs):
    return (np.mean(costs), np.median(costs), np.max(costs))

# Apply to 'Cost' column
mean_cost, median_cost, max_cost = cost_summary(df['Cost'])

print(f"Mean: {mean_cost}, Median: {median_cost}, Max: {max_cost}")
```

```
Mean: 12504.5, Median: 15000.0, Max: 25000.0
```

Dictionary

```
import pandas as pd

# Add and map vehicle type
types = ['SUV', 'Sedan', 'Convertible', 'Hatchback', 'Wagon']
df['vehicle_type'] = [types[i % 5] for i in range(len(df))]
map_dict = {'SUV': 'Sport Utility', 'Sedan': 'Sedan', 'Convertible': 'Convertible', 'Hatchba
df['vehicle_type_standardized'] = df['vehicle_type'].map(map_dict)

# Dictionary for column-wise statistics
stats = {
    'Cost': df['Cost'].agg(['mean', 'median', 'min', 'max']).to_dict(),
    'Odometer': df['Odometer'].agg(['mean', 'median', 'min', 'max']).to_dict()
}

# Output results
print("=== Mapped Vehicle Types (first 5) ===")
print(df[['vehicle_type', 'vehicle_type_standardized']].head())

print("\n=== Column-wise Statistics Dictionary ===")
for column, stat in stats.items():
    print(f"{column}: {stat}")
```

```
=== Mapped Vehicle Types (first 5) ===
  vehicle_type vehicle_type_standardized
0          SUV             Sport Utility
1        Sedan                     Sedan
2  Convertible               Convertible
3    Hatchback                 Hatchback
4        Wagon                     Wagon

=== Column-wise Statistics Dictionary ===
Cost: {'mean': 12504.5, 'median': 15000.0, 'min': 0.0, 'max': 25000.0}
Odometer: {'mean': 124920.90583631671, 'median': 150000.0, 'min': 50000.0, 'max': 200000
```

PART 6: SUMMARY AND SUBMISSION

Summary of Data Cleaning:

- **Total rows before cleaning**: `total_rows_before`
- **Total rows after removing duplicates**: `total_rows_after`
- **Rows removed**: `rows_removed` → This helps track how many duplicate entries were eliminated.
- **Missing values before**: `missing_values_before`
- **Missing values after filling**: `missing_values_after`
- **Total missing values filled**: `missing_values_filled` → Missing values in `Cost` and `Odometer` were filled with `0`, and `FuelType` with `"Unknown"`.

Insights:

1. **Highest Cost** → The maximum car cost was found using: `df['Cost'].max()`
2. **Most Frequent Fuel Type** → Identified using: `df['FuelType'].mode()[0]`
3. **Average Odometer for High-Cost Cars** → For cars costing more than 20000, average odometer was calculated with: `df[df['Cost'] > 20000]['Odometer'].mean()`

IMPORTANCE OF DATA WRANGLING:

While doing this assignment, I realized how crucial data wrangling is for working with real data. The dataset had problems like missing values, repeated rows, and inconsistent formats in columns like `Cost`, `Odometer`, and `FuelType`. If I hadn't fixed these issues, my analysis would have been wrong.

To clean the data, I removed duplicate rows, filled missing values with defaults (like `"Unknown"` or `0`), and made categories consistent. For example, I fixed 1,499 missing values and removed 49 duplicate rows.

This made the data easier to work with, allowing me to quickly find insights, like the most expensive car, the most common fuel type, and the average mileage of high-cost cars. Without data wrangling, these insights would have been unclear.